Task: Implement a Fake Payout Provider for Testing

I am working on a NestJS backend project called **SportBooking**.

The project already has a payment module for receiving money from customers. I now want to implement the **Payout** module, but **ONLY for testing purposes**. There is no real bank API or payout provider yet.

Goal

Implement a fake payout provider that simulates the behavior of a real bank.

The architecture should be designed so that in the future I can replace the fake provider with a real bank provider without changing the business logic.

Requirements

1. Create a Provider abstraction

Create a provider interface similar to the existing payment providers.

Example:

- `PayoutProvider`
  - `createTransfer()`

Future implementations:

- FakeProvider
- ManualProvider
- BankProvider (future)

The business layer should only depend on the interface.

---

2. Fake Provider

Implement `FakeProvider`.

When `createTransfer()` is called:

- Generate a fake provider transaction id (UUID)
- Return immediately with

```
status = PROCESSING
```

Do NOT return SUCCESS immediately because real banks are asynchronous.

---

3. Payout Status

Support at least:

```
PENDING
PROCESSING
SUCCESS
FAILED
```

---

4. Simulate asynchronous processing

After returning PROCESSING:

Wait around 3 seconds.

Then simulate the bank sending a webhook.

Do NOT directly update the database from FakeProvider.

Instead call the application's webhook endpoint just like a real provider would.

Example:

```
POST /webhooks/payout
{
    providerTransactionId,
    payoutId,
    status: "SUCCESS"
}
```

---

5. Random Result

The fake provider should simulate different outcomes.

Example probabilities:

- 80% SUCCESS
- 20% FAILED

Please make the probabilities configurable.

---

6. Optional Testing Mode

Allow forcing the result.

Example request:

```
{
    "amount":500000,
    "bankCode":"VCB",
    "forceResult":"FAILED"
}
```

Supported values:

```
SUCCESS
FAILED
TIMEOUT
RANDOM
```

If not provided, use RANDOM.

---

7. Timeout Simulation

Support TIMEOUT mode.

In this case:

- no webhook is sent
- payout remains PROCESSING

This allows testing retry jobs.

---

8. Webhook Controller

Create:

```
POST /webhooks/payout
```

The webhook should:

- verify payload
- locate payout by providerTransactionId

- update status
- record provider response
- save updatedAt

---

9. Idempotency

Webhook must be idempotent.

If SUCCESS has already been processed, ignore duplicated webhook requests.

Do not update the same payout twice.

---

10. Retry Ready

Design the code so that a future scheduled job can retry payouts stuck in PROCESSING.

Do not implement the scheduler now, only make the architecture ready.

---

11. Logging

Log every step.

Example:

```
Payout Created

↓

Provider Called

↓

PROCESSING

↓

Webhook Received

↓

SUCCESS
```

---

12. Code Quality

Use:

- NestJS best practices
- Dependency Injection
- Interface-based design
- SOLID principles
- Clean Architecture where possible

Avoid putting provider logic inside the service.

---

13. Suggested Folder Structure

```
src/payouts
│
├── controllers
│     payout.controller.ts
│
├── services
│     payout.service.ts
│
├── providers
│     payout-provider.interface.ts
│     fake.provider.ts
│     manual.provider.ts
│
├── webhooks
│     payout-webhook.controller.ts
│
├── dto
│
├── entities
│
└── enums
```

---

14. Important

Please inspect my existing project structure first.

Reuse existing coding conventions.

If there is already a Payment Provider pattern, follow the same style.

Do not duplicate existing abstractions.

Generate production-quality code instead of pseudo-code.

Explain every new file you create and why it is needed.